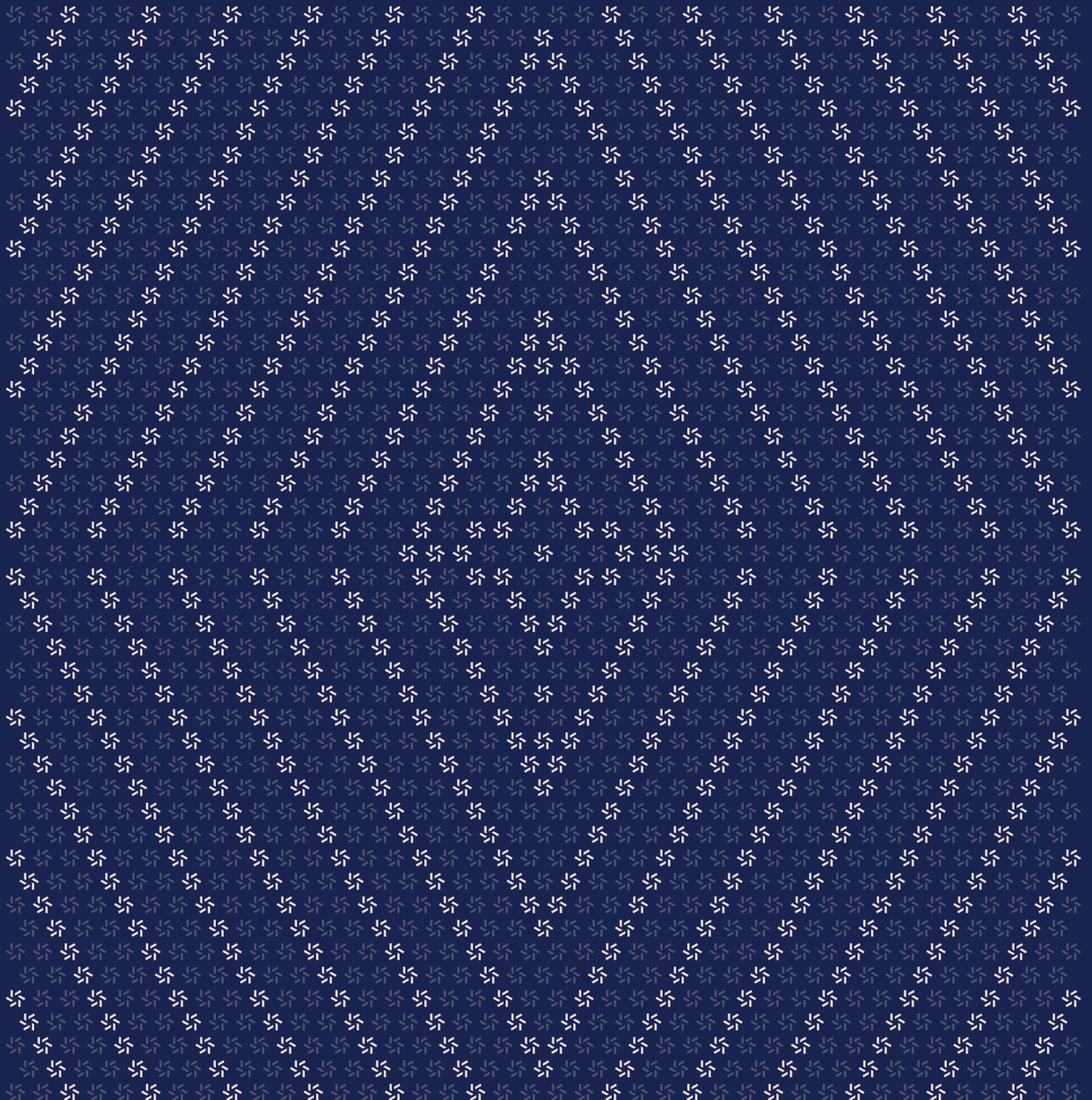


February 10, 2026

EdgeX V2

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About EdgeX V2	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. The <code>_checkValidTransition</code> function allows the value to become negative	11
3.2. Liquidator must immediately take loss when liquidating net negative position	14
3.3. Collateral balance should not be deemed as an available amount to be removed	17
3.4. The permit deposit logic allows approvals to be consumed incorrectly	20
3.5. The <code>DelistSettle</code> transaction does not handle accumulated funding	23
3.6. Deleveraging does not require a signature from the deleverager	24
3.7. Centralization risks due to checks bypassed by owner calls	25
3.8. High degree of centralization risk on the owner and sequencer	28

3.9.	Time-weighted average-price suborders are broadly unconstrained	30
3.10.	The RemoveIsolatedMargin transaction does not recognize collateral added through funding	35
3.11.	Using the median oracle price's timestamp for the aggregate timestamp leads to undesirable edge-case behavior	36
3.12.	Zero orderHash case for order signature check may cause type confusion	39
3.13.	Changing the mode of risk segment is nonatomic	41
3.14.	The AddIsolatedMargin transaction can be processed for delisted assets	43
3.15.	Unused asset ID of collateral	44
3.16.	The deposit workflow does not support rebasing or fee-on-transfer tokens	45
3.17.	Funding calculation rounding should always round against the user	47
4.	Discussion	48
4.1.	If re-deployed, the Create2Factory may be a reputational hazard	49
4.2.	Keeping assets sorted by asset ID could be a worthwhile optimization	50
4.3.	Inconsistent hash ordering between AddIsolatedMargin and RemoveIsolatedMargin	50
5.	System Design	51
5.1.	Component: Transaction Batchers	52
6.	Assessment Results	53
6.1.	Disclaimer	54

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for EdgeX from January 15th to February 4th, 2026. During this engagement, Zellic reviewed EdgeX V2's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the contract verify signatures correctly?
 - Are there any bugs that may result in access control bypasses?
 - Are there any arithmetic bugs (e.g. overflow/underflow, precision loss)?
 - Are there any logical flaws or edge cases that could brick the contract?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Use of fee-on-transfer or rebasing tokens as the vault asset
- Malicious transaction batcher activity, including front-running or sandwich attacks
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

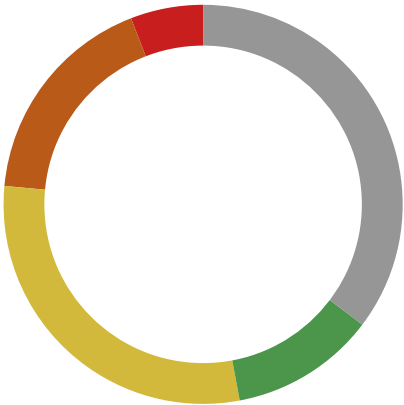
During our assessment on the scoped EdgeX V2 contracts, we discovered 17 findings. One critical issue was found. Three were of high impact, five were of medium impact, two were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of EdgeX in the Discussion section ([4. 7](#)).

Based on the number of severe findings uncovered during the audit, it is our opinion that the project is not yet ready for production. We strongly advise a comprehensive reassessment before deployment to help identify any potential issues or vulnerabilities introduced by necessary fixes or changes. We also recommend adopting a security-focused development workflow, including (but not limited to) augmenting the repository with comprehensive end-to-end tests that achieve 100% branch coverage using any common, maintainable testing framework, thoroughly documenting all function requirements, and training developers to have a security mindset while writing code.

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	3
Medium	5
Low	2
Informational	6



2. Introduction

2.1. About EdgeX V2

EdgeX contributed the following description of EdgeX V2:

EdgeX V2 is a decentralized perpetual contract exchange (Perpetual DEX) that enables trust-less derivatives trading with a hybrid off-chain/on-chain architecture. The protocol allows users to trade synthetic assets (perpetual futures) off-chain for efficiency while maintaining on-chain security through smart contract settlement.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

EdgeX V2 Contracts

Type	Solidity
Platform	EVM-compatible
Target	edgex-perpetual-contract
Repository	https://github.com/edgex-org/edgex-perpetual-contract ↗
Version	09dccf0604099a024422fd9f32c9e0b6ecbbcf1f
Programs	src/**/* .sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 4.4 person-weeks. The assessment was conducted by two consultants over the course of 3.2 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jinseo Kim
✈ Engineer
jinseo@zellic.io ↗

Kuilin Li
✈ Engineer
kuilin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

January 15, 2026	Kick-off call
-------------------------	---------------

January 15, 2026	Start of primary review period
-------------------------	--------------------------------

February 4, 2026	End of primary review period
-------------------------	------------------------------

3. Detailed Findings

3.1. The `_checkValidTransition` function allows the value to become negative

Target	PositionManager		
Category	Business Logic	Severity	Critical
Likelihood	Medium	Impact	Critical

Description

Every time a position changes, the checks defined in the `_checkValidTransition` function run in order to ensure that the new position is not more risky:

```
function _checkValidTransition(
    AccountPosition memory initPosition,
    AccountPosition memory updatedPosition,
    bool isLiquidation
) internal view {
    (int256 updatedTotalValue, int256 updatedTotalRisk)
    = getAccountPositionStatus(updatedPosition);
    if (updatedTotalRisk <= updatedTotalValue * FXP_32_ONE) {
        // Position is healthy.
        // For liquidation: check sign change to prevent over-closing.
        // For normal trades: no restriction, user can flip position direction.
        if (isLiquidation) {
            _checkSmallerInSyntheticHoldings(initPosition, updatedPosition,
            true);
        }
        return;
    }
    if (updatedTotalRisk == 0) {
        return;
    }
    (int256 initTotalValue, int256 initTotalRisk)
    = getAccountPositionStatus(initPosition);

    // High risk - full check: position reduction only, no position increase,
    // no new asset IDs.
    _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, false);

    // total_value / total_risk must not decrease.
    // tv0 / tr0 <= tv1 / tr1 iff tv0 * tr1 <= tv1 * tr0.
    // tv is 96 bit, tr is 128 bit, so tv*tr fits in 224 bits.
    // Since tv can be negative, adding SIGNED_COMPARISON_OFFSET to each side.
```

```
if (
    (SIGNED_COMPARISON_OFFSET + initTotalValue * updatedTotalRisk)
    > (SIGNED_COMPARISON_OFFSET + updatedTotalValue * initTotalRisk)
) {
    revert("PM: val/risk dec");
}
if (initTotalRisk == 0) {
    if (updatedTotalValue < initTotalValue) {
        revert("PM: val dec");
    }
}
}
```

There is a short circuit here, where if `updatedTotalRisk == 0`, the function exits without checking the total value. The risk is calculated in the `_getAccountPositionStatusInner` function:

```
function _getAccountPositionStatusInner(AccountPosition memory position)
    internal view returns (int256, int256) {
    // [...]
    int256 totalRiskRep = 0;
    uint256 len = position.assets.length;
    for (uint256 i = 0; i < len; i++) {
        PositionAsset memory asset = position.assets[i];
        uint64 assetId = asset.packed.getAssetId();
        // [...]
        int256 absBalance = asset.balance < 0 ? -asset.balance :
asset.balance;
        (int256 riskFactor, bool exist) = generalConfig.getRiskFactor(assetId,
absBalance, oraclePrice);
        // [...]
        int256 absValueRep = absBalance * oraclePrice;
        int256 riskRep = absValueRep * riskFactor;
        // [...]
        totalRiskRep += riskRep;
        unchecked {
            ++i;
        }
    }
    return (totalValueRep, totalRiskRep);
}
```

The risk of each position is strictly nonnegative, because it is the product of a nonnegative `absBalance` with a nonnegative `riskFactor`. This means that if the total risk is zero, then the position must have no assets in `position.assets`. However, the collateral balance is not considered in this total risk.

This means that a position with zero assets and negative collateral balance will skip the high-risk checks. Since such a position cannot be profitably liquidated and represents pure bad debt, it should not pass `_checkValidTransition`.

Impact

Although some transactions, such as withdrawal and transfer, ensure that the collateral balance must be positive, this is not a guarantee. For instance, the deleverage transaction calls `updatePositionInDict` on the deleverager without checking that the `deleveragerCollateralDelta` does not reduce the deleverager's collateral below zero. Furthermore, in Finding [3.3](#), we described how the positive collateral balance requirement may be an unnecessarily arbitrary requirement.

We did not fully assess the impact of this finding due to its dependence on economic factors, but, if a position is updated in such a way that it ultimately becomes this zero-asset negative-collateral case, then it would immediately become bad debt on the protocol.

Recommendations

Update this short circuit to consider the case where `updatedTotalValue` is negative. Alternatively, consider other designs alongside the remediation for Finding [3.3](#).

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [46385b82](#).

3.2. Liquidator must immediately take loss when liquidating net negative position

Target	PositionManager		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

If a position of a user (cross-account position or isolated position) is unhealthy, the position can be liquidated; without the approval of the user ("liquidatee"), their position can be reduced as part of processing an order from another user ("liquidator").

Every change on position (balance of synthetic or collateral), including liquidation, undergoes the checks defined in the `_checkValidTransition` function:

```
function _checkValidTransition(
    AccountPosition memory initPosition,
    AccountPosition memory updatedPosition,
    bool isLiquidation
) internal view {
    (int256 updatedTotalValue, int256 updatedTotalRisk)
    = getAccountPositionStatus(updatedPosition);
    if (updatedTotalRisk <= updatedTotalValue * FXP_32_ONE) {
        // Position is healthy.
        // For liquidation: check sign change to prevent over-closing.
        // For normal trades: no restriction, user can flip position direction.
        if (isLiquidation) {
            _checkSmallerInSyntheticHoldings(initPosition, updatedPosition,
            true);
        }
        return;
    }
    if (updatedTotalRisk == 0) {
        return;
    }
    (int256 initTotalValue, int256 initTotalRisk)
    = getAccountPositionStatus(initPosition);

    // High risk - full check: position reduction only, no position increase,
    // no new asset IDs.
    _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, false);
}
```

```
// total_value / total_risk must not decrease.
// tv0 / tr0 <= tv1 / tr1 iff tv0 * tr1 <= tv1 * tr0.
// tv is 96 bit, tr is 128 bit, so tv*tr fits in 224 bits.
// Since tv can be negative, adding SIGNED_COMPARISON_OFFSET to each side.
if (
    (SIGNED_COMPARISON_OFFSET + initTotalValue * updatedTotalRisk)
    > (SIGNED_COMPARISON_OFFSET + updatedTotalValue * initTotalRisk)
) {
    revert("PM: val/risk dec");
}
if (initTotalRisk == 0) {
    if (updatedTotalValue < initTotalValue) {
        revert("PM: val dec");
    }
}
}
```

When liquidation takes place, this means the following:

1. If the value (total value of the position) exceeds the risk (sum of absolute balances of synthetics multiplied with the risk factor of the asset), a sign of any synthetic balance must not be changed.
2. If the risk is equal to zero, there are no additional checks.
3. Otherwise,
 - a. the absolute balance of any synthetic balance must not increase;
 - b. $\frac{\text{initial value}}{\text{initial risk}} \leq \frac{\text{updated value}}{\text{updated risk}}$; and
 - c. when the risk is equal to zero, the value must not decrease.

The second condition in the third point tries to ensure the value divided by the risk (known as health factor) increases. This check can become problematic when the position with the net negative value is liquidated.

Typically, the liquidation is processed under the fair price (i.e., the price that does not change the total value much). If the total value of a position is positive, the health factor would increase after the liquidation, considering that the total risk always decreases after the liquidation (due to the first condition in the third point) and the total value would not be changed much.

If the total value of a position is negative, the health factor decreases when the risk decreases; since it is ensured that the risk decreases, the liquidation cannot be done unless the total value increases, involving the liquidator taking loss immediately (by buying the position of the liquidatee at a higher cost than the fair price).

Note that the second point does not mitigate this issue because a user may have multiple synthetic assets, which should be liquidated individually. Additionally, we believe this second point should be removed; see Finding [3.1](#).

Impact

A liquidator does not have economic incentive to liquidate a user who possesses a position with negative value, since the protocol requires the liquidator to take loss immediately to perform the liquidation. Not performing liquidation on time may intensify the insolvency of protocol.

Recommendations

Consider refactoring the business logic of liquidation.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [1d5ebf40](#). This change adds a signature check and whitelist to the deleverage transaction, while removing the oracle price check and requirement. EdgeX noted the following:

We'd like to clarify the intended design and explain how the latest code changes address this.

1. When `TotalValue < 0`, the protocol routes exclusively to ADL, not regular liquidation. (extremely rare occurrence in practice)
2. The ADL party (deleverager) bearing a loss is expected behavior by design.
3. Regarding the `_checkValidTransition` constraint: the finding correctly identifies that ADL at exact oracle price fails the health ratio check when `TotalValue < 0`, because reducing risk without improving `TotalValue` worsens the tv/tr ratio. This has been addressed in the changes: the oracle price peg in `DeleverageTransLib` has been removed, allowing the off-chain sequencer to submit ADL transactions at a price worse than oracle for the deleveragable account, which allows `_checkValidTransition` ratio check to be passed.

3.3. Collateral balance should not be deemed as an available amount to be removed

Target	PositionManager, TransferTransLib, WithdrawalTransLib		
Category	Coding Mistakes	Severity	High
Likelihood	High	Impact	High

Description

The Withdrawal and Transfer transactions deduct collateral balance from a position. Similarly, AddIsolatedMargin and RemoveIsolatedMargin transactions transfer collateral balance between isolated and cross-account positions. Since these transactions mutate the collateral balance, they first check that the amount of collateral to be removed is greater than the current balance in the position:

```
library WithdrawalTransLib {
    function process(
        Withdrawal memory self,
        IPositionManager position,
        IAccountManager accountManager,
        ILoadingZone loadingZone,
        ISignerManager signerManager
    ) public {
        // (...)
        if (position.getCollateralBalanceByAccountId(uint64(self.from)) <
            self.amount) {
            revert("WithdrawalTransLib: insufficient balance");
        }
        // (...)
    }
    // (...)
}
```

```
library TransferTransLib {
    function process(Transfer memory self, IPositionManager position,
        ISignerManager signerManager) public {
        // (...)
        if (position.getCollateralBalanceByAccountId(self.from) < self.amount)
        {
            revert("TransferTransLib: insufficient balance");
        }
    }
}
```

```
    }  
    // (...)  
  }  
}
```

```
contract PositionManager is IPositionManager, Initializable,  
    OwnableUpgradeable {  
    // (...)  
    function addIsolatedMargin(uint64 assetId, uint64 accountId,  
        int256 collateralAmount)  
        external  
        onlyTransactionProcessorOrOwner  
    {  
        // (...)  
        require(acc.collateralBalance >= collateralAmount, "PM: bal low");  
        // (...)  
    }  
    // (...)  
}
```

```
contract PositionManager is IPositionManager, Initializable,  
    OwnableUpgradeable {  
    // (...)  
    function getMaxRemovableIsolatedMargin(uint64 accountId, uint64 assetId)  
    public view returns (int256) {  
        // (...)  
        if (maxRemovable > asset.isolatedCollateral) {  
            maxRemovable = asset.isolatedCollateral;  
        }  
        // (...)  
    }  
    // (...)  
}
```

However, this check is inconsistent with the lack of checks in the other transactions. For instance, the Deleverage and Trade transactions can eventually call `updatePositionInDict` with a collateral delta that reduces the collateral balance past zero, if the risk checks permit it.

In abstract, these checks and Finding 3.1 [↗](#) seem to imply an alternative synthetic asset design where the `collateralBalance` collateral amount is held in order to backstop the existence of the synthetic positions. However, this design is not the one that these contracts actually implement. Instead, the `collateralBalance` is directly treated as an asset, one that can be traded for the other synthetic assets.

In more concrete terms, assume that there exists a synthetic asset that is freely convertible to and

from the actual collateral asset, so there exists a very liquid market for buying and selling the asset at the unit price. If a user's Withdrawal transaction fails due only to this balance check, they can get around it by immediately short-selling the synthetic asset, receiving collateral in return. This action would not change anything about the actual economic position or its risks, but it would enable this withdrawal.

Impact

This balance check is inconsistent with the design of the collateral balance implemented in the rest of the system, which may lead to unexpected reverts.

Recommendations

Consider removing this check. Note that Finding [3.1](#) should be remediated with this one in mind.

Remediation

This issue has been acknowledged by EdgeX, and fixes were implemented in the following commits:

- [e4f2419c](#)
- [e947a89b](#)
- [08cd3b0f](#)

These changes implement the zero-check removal. We did not review the broader suitability of this beyond its application to resolving this finding.

3.4. The permit deposit logic allows approvals to be consumed incorrectly

Target	LoadingZone		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	High

Description

The `depositWithPermit` function allows a user to use a permit and deposit the permitted tokens to their own account:

```
function depositWithPermit(address user, uint256 amount, uint256 deadline,
    uint8 v, bytes32 r, bytes32 s) external {
    require(amount > 0, "LoadingZone: amount must be greater than 0");
    require(address(edgeVault) != address(0), "LoadingZone: edgex vault not set");

    // Try permit to EdgexVault - if it fails (e.g., frontrun), continue anyway
    // since allowance may already be set
    try IERC20Permit(address(depositToken)).permit(user, address(edgeVault),
        amount, deadline, v, r, s) {} catch {}

    balances[user] += amount;
    // EdgexVault pulls tokens from user
    edgeVault.depositFrom(user, address(depositToken), amount);

    emit TokenDeposited(user, amount, balances[user]);
}
```

This function does not revert when it receives an invalid permit. This is a standard design because otherwise, anyone could front-run the call to `depositWithPermit` call and cause it to revert, because they can submit the permit themselves, and therefore the second call to `permit` will reuse the same nonce. However, an important caveat of this design is that it implies that any approval the vault has is because of a specific user intent to deposit the approved funds into their own account.

The `depositTo` function contradicts this assumption:

```
function depositTo(address to, uint256 amount) external {
    require(to != address(0), "LoadingZone: invalid to address");
    require(amount > 0, "LoadingZone: amount must be greater than 0");
    require(address(edgeVault) != address(0), "LoadingZone: edgex vault not set");
}
```

```
set");
balances[to] += amount;
// EdgexVault pulls tokens from user (user must approve EdgexVault)
edgexVault.depositFrom(msg.sender, address(depositToken), amount);

emit TokenDeposited(to, amount, balances[to]);
}
```

In order to use `depositTo` to deposit tokens on behalf of a different recipient `to`, a user first gives the vault some approval and then calls the function. In between those two function calls, the assumption is broken.

Impact

Normal users, with EOAs that do not contain code, cannot safely use `depositTo` without risk of being griefed. This is because the approval and `depositTo` calls must be two separate transactions, and, if an attacker has control over transaction ordering, they can insert a transaction in the middle of them that calls `depositWithPermit` with a garbage permit. This will cause the deposit to land in the victim's own account, rather than the address `to` that they meant to deposit into.

Additionally, any normal user who mistakenly gives the vault unlimited approval will be unable to safely use any other protocol that manipulates the same token, because an attacker can front-run them with a `depositWithPermit` that instead deposits them here.

Note that in both the above hypotheticals, the user does not sign any permit themselves.

Recommendations

One way to resolve this issue is to require a second non-ERC-2612 signature that confirms the intended destination for the coins. This may be implemented by the `LoadingZone`, or it can also use a central permit protocol such as the `Uniswap Permit2`, whose `SignatureTransfer` module only allows the spender to consume permits written for it and which allows the addition of arbitrary witness data into the signature. The intended recipient can be encoded into the witness data.

Another way to resolve this is to maintain a parallel permit nonce status. The function signature would have to be modified to add the nonce, since there is no way to retrieve the nonce if the permit was already used. But, with the nonce, the signature could be verified and then marked as used even if the call to the ERC-2612 permit function reverts.

Alternatively, if the impact of front-running permits is negligible, the try-catch statement could just be removed.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [1db7e235](#). This change implements the third recommendation stated above. The risk of permits being frontrun to cancel calls to `depositWithPermit` was acknowledged by EdgeX.

3.5. The DelistSettle transaction does not handle accumulated funding

Target	PositionManager		
Category	Coding Mistakes	Severity	Medium
Likelihood	High	Impact	Medium

Description

The contract stores the last applied funding index of a position. When the position is changed, it is responsible for fetching the new funding index and applying the accumulated funding to the collateral balance.

The DelistSettle transaction provides a way to settle the position for whose asset is delisted. The `forceSettlePosition` function is responsible for handling this process by reducing the synthetic balance from the position and adding/subtracting the collateral balance accordingly. However, it does not handle the accumulated funding.

Impact

Accumulated funding will not be applied when the position of a delisted asset is settled.

Recommendations

Consider adding the business logic to apply accumulated funding on the DelistSettle transaction.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [1870093b](#).

3.6. Deleveraging does not require a signature from the deleverager

Target	DeleverageTransLib		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	Medium

Description

Deleverage is the transaction that sells the deleverageable position to the deleverager provided that the deleverageable position is risky. The Deleverage transaction does not verify the signature of the deleverager.

Impact

A user may undesirably buy a position if the sequencer designates the user as the deleverager for a Deleverage transaction.

Recommendations

Consider checking the signature of the deleverager on the Deleverage transaction. Alternatively, if one central account is used for these deleveraging actions, then consider adding a configuration entry for this account with the explicit indication that the sequencer can control their position.

Remediation

This issue has been acknowledged by EdgeX. EdgeX notified us that they believe it is not practical on their practice to require the signature of deleverager on the transaction. Instead, EdgeX implemented a fix that verifies the signature of a deleverage operator account (a new privileged role in the protocol) in commit [1d5ebf40](#). This signature double-checks that the deleverage operation was authorized in conjunction with the trusted sequencer's inclusion of the transaction - note that by itself, it lacks a nonce and can be replayed.

3.7. Centralization risks due to checks bypassed by owner calls

Target	AccountManager, Funding, Oracle, TwapOrderManager, PositionManager, SignerManager		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

Normally, for nearly all transactions, the calls to the inner logic contracts come through the PositionManager or the TransactionProcessor. However, many of the functions they call can also be called directly by the owner of the contract, since they have the onlyTransactionProcessorOrOwner or onlyPositionManagerOrOwner modifiers. These direct calls skip certain checks in an inconsistent way.

For instance, in the DelistSettle transaction, the forceSettlePosition function is called with its last parameter set to the current syntheticBalance:

```
library DelistSettleTransLib {
    function process(
        // [...]
    ) internal {
        // [...]
        int256 syntheticBalance = position.getSyntheticBalance(self.accountId,
            self.syntheticAssetId);
        // [...]
        position.forceSettlePosition(self.accountId, self.syntheticAssetId,
            collateralDelta, -syntheticBalance);
    }
}
```

```
function forceSettlePosition(uint64 accountId, uint64 assetId,
    int256 collateralDelta, int256 syntheticDelta)
    external
    onlyTransactionProcessorOrOwner
{
    uint128 key = _syntheticPositionKey(accountId, assetId);
    PositionAsset storage existing = syntheticPositions[key];
    require(existing.packed.getAssetId() != 0, "PM: pos not found");
    // [...]
    existing.balance += syntheticDelta;
}
```

```
// [...]  
accountManager.removeAsset(accountId, assetId, marginMode);  
delete syntheticPositions[key];  
// [...]  
}
```

This function is `onlyTransactionProcessorOrOwner`, so the owner can conceivably directly call it. Based on the function arguments, the `syntheticDelta` argument appears to be the amount that the synthetic asset changed by. However, although the `syntheticDelta` parameter is added to `existing.balance`, ultimately this key is deleted in the same function. So this argument is functionally unused except for an overflow check. If the owner calls this function expecting a different result if they assign this parameter to something other than the existing size, then the logic would be incorrect.

Similarly, the `updateFundingIndex` function is `onlyTransactionProcessorOrOwner`, but the `maxFundingRate` check exists in the `FundingLib`, not the `Funding` contract. So if the owner directly calls it, it will bypass the rate limit.

These are implemented in an inconsistent way because in the same contracts, there is still an `onlyTransactionProcessor` modifier that excludes the owner from calling those functions.

Impact

Since this audit assumes that the owner is trusted and calls functions carefully, and since the owner already has broad control over the protocol (see Finding [3.8](#) ↗), this centralization is an expected part of the design of the system.

Even though it is part of the design, it is still important to note that the owner-callable functions as they are currently designed can lead to significant footguns.

Recommendations

We recommend carefully reviewing the owner-facing interface to ensure that all of the owner-callable functions can be used without footguns, in a way that aligns with the intent of those future calls even if the owner is less careful than the current design expects.

Alternatively, we recommend removing the `onlyTransactionProcessorOrOwner` and `onlyPositionManagerOrOwner` modifiers, in favor of disallowing the owner from calling internal functions directly. Instead, in order to manually resolve issues, the owner could have the ability to directly mutate balances and positions without the standard checks. This would implement the same level of permissivity more directly, which is both more self-documenting of the centralization risks present here and safer against incorrect logic due to partially executing logic that expects a caller to have already performed certain checks or actions.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [48c138cd](#).

3.8. High degree of centralization risk on the owner and sequencer

Target	PositionManager		
Category	Protocol Risks	Severity	High
Likelihood	Low	Impact	Medium

Description

The owner of these contracts has the broad ability to manage them, and, as a result, it represents a central point of failure if the owner is ever compromised.

For instance, the owner can replace the sequencer to ensure that only their transactions go through and then submit false funding ticks to liquidate all of the accounts into one the owner controls. Although there is a limit on how fast the funding snapshot can change, the owner can call into or replace the GeneralConfig contract in order to change that limit without any delay. Another avenue for draining the protocol that is available to the owner is via price manipulation, replacing the oracles with falsified ones in order to liquidate positions.

Another aspect of centralization risk is the sequencer itself. Except for the first steps of the deposit flow, end users interact with this system only through the sequencer, which cannot be interacted with on chain. So, if the sequencer ever goes down indefinitely, users will not be able to change their positions or withdraw any assets.

Impact

These contracts do not meaningfully limit the blast radius of a compromised owner account — within one transaction, a compromised owner can potentially drain all of the assets held by the contract.

Additionally, users rely on the centralized sequencer to do nearly all of the actions they can do. If the sequencer ever goes down, then users will not be able to settle any of their positions.

Recommendations

We recommend implementing a timelock on owner activities so that users can react accordingly before any proposed significant change actually occurs.

Additionally, we recommend implementing an escape hatch for users to execute some transactions, even if it is just the offboarding transactions, without the sequencer. This would guard against the sequencer censoring any or all users through not sequencing their actions.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

The owner and sequencer roles are and will be governed off-chain through multi-signature schemes, limiting the blast radius of any single compromised key. We recognize the residual centralization risk and will continue to evolve operational controls around these privileged roles.

3.9. Time-weighted average-price suborders are broadly unconstrained

Target	LimitOrderLib, TwapOrderManager		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

Time-weighted average-price (TWAP) orders are special orders where the order that the end user submits is a parent TwapOrder struct, which does not constrain the price. Then, some off-chain entity generates normal LimitOrder orders that reference the parent TwapOrder, partially filling it as time goes on.

This process places a lot of trust on the sequencer to execute the TWAP orders correctly. Here is a nonexhaustive list of important details that the contract code does not constrain.

Suborder signatures are not checked

The validateOrderAndUpdateFulfillmentWithTwap function includes a comment explicitly skipping the signature verification for TWAP suborders:

```
function validateOrderAndUpdateFulfillmentWithTwap(
    // [...]
) public {
    // [...]
    if (isTwapSubOrder(self)) { // audit: return self.parentOrderHash !=
        bytes32(0);
        // TWAP sub-order validation with security guardrails
        require(
            address(twapOrderManager) != address(0), "LimitOrderLib: TWAP
            order manager required for sub-orders"
        );

        // Get parent order for binding validation
        TwapOrder memory parent
        = twapOrderManager.getParentOrder(self.parentOrderHash);
        require(parent.orderHash != bytes32(0), "LimitOrderLib: parent order
        not found");

        // Critical security validations
```

```
        require(self.base.signer == parent.base.signer, "LimitOrderLib: signer mismatch");
        require(self.base.accountId == parent.base.accountId, "LimitOrderLib: account mismatch");
        require(self.assetIdSynthetic == parent.assetIdSynthetic, "LimitOrderLib: synthetic asset mismatch");
        require(self.assetIdCollateral == parent.assetIdCollateral, "LimitOrderLib: collateral asset mismatch");
        require(self.isBuyingSynthetic == parent.isBuyingSynthetic, "LimitOrderLib: direction mismatch");

        // Index bounds validation
        require(
            self.subOrderIndex >= 0 && self.subOrderIndex <
            parent.subOrderCount,
            "LimitOrderLib: invalid sub-order index"
        );

        require(block.timestamp <= parent.base.expirationTimestamp, "LimitOrderLib: parent order expired");

        // Enforce parent-level fulfillment cap (critical security)
        twapOrderManager.updateParentFulfillment(self.parentOrderHash, updateAmount);

        // No signature verification needed - inherits from parent
    } else // [...]
    }
```

This logic checks whether most of the fields in the TWAP suborder match the details committed by the parent. However, an important field that it does not check is that the price implied by the synthetic and collateral amounts is a fair price. Because this field is unconstrained, even though the suborder may have been generated by a trusted entity, the sequencer can mutate it in order to cause the order to be filled at an incorrect price.

Another detail of this is that there exists a helper function `_verifyTwapSubOrder` that verifies the signature of the parent order of a suborder. This function, however, is unused. It is called in only one location, in the four-argument form of `verifyOrderSignature`, but this four-argument `verifyOrderSignature` is never called by any other on-chain code.

Suborder count does not limit the number of suborders

TWAP parent orders have a positive `subOrderCount` field, which contains the expected number of suborders for this order. This field is checked in the same `validateOrderAndUpdateFulfillmentWithTwap` function:

```
// Index bounds validation
require(
    self.subOrderIndex >= 0 && self.subOrderIndex < parent.subOrderCount,
    "LimitOrderLib: invalid sub-order index"
);
```

From the name of the field, it appears to limit the number of suborders that can refer to this parent. However, this is the only time both it and the subOrderIndex field of any suborder are checked. This means that the sequencer can repeat suborders with the same index any number of times — as long as any index is supplied that is within the range, it will pass this bounds check.

The user's maximum fee is not enforced

All limit orders have an amountFee field, which states the maximum amount of fee that the user is willing to pay for the order:

```
struct LimitOrder {
    // (...)
    uint256 amountFee;
    // (...)
}
```

This field is verified in the signature and enforced in the business logic for non-TWAP orders:

```
function getLimitOrderHash(LimitOrder memory order, address verifyingContract)
    internal view returns (bytes32) {
    bytes32 structHash = keccak256(
        abi.encode(
            // (...)
            order.amountFee,
            // (...)
        )
    );
    return structHash.getTypedDataHash(verifyingContract);
}
```

```
function validateLimitOrderFairness(
    LimitOrder memory self,
    int256 actualCollateral,
    int256 actualSynthetic,
    uint256 actualFee
) public pure {
    // (...)
}
```

```
require(
    feeInt * amountCollateral <= amountFee * actualCollateral,
    "LimitOrderLib: fee * amountCollateral > collateral * amountFee"
);
// (...)
}
```

However, TWAP orders do not have the field stating the maximum amount of fees the submitter is willing to pay, and the contracts do not have any business logic that limits the amount of fees for the TWAP order.

Therefore, the sequencer may declare an arbitrary amount of fee for processing a TWAP order.

Time-weighted averaging detail is not enforced for TWAP order

Internally, when the sequencer processes a TWAP order, it generates the suborders corresponding to the part of the order. These suborders should be generated and submitted abiding by instructions from the user on the way the suborders should be placed over time.

However, such instructions are not enforced since the TWAP order does not contain the relevant fields. Specifically, it only provides the asset, total size of the order, and the number of suborders:

```
struct TwapOrder {
    OrderBase base;
    uint64 assetIdSynthetic;
    uint64 assetIdCollateral;
    bool isBuyingSynthetic;
    int256 totalSize;
    uint256 subOrderCount;
    bytes32 orderHash;
}
```

So the sequencer may place the suborder of the TWAP order arbitrarily, violating any actual time-weight averaging instructions from the user.

Impact

The TWAP order feature is broadly centralized and relies on the correct operation of the off-chain infrastructure in order to maintain correctness. The on-chain TWAP order code does not meaningfully constrain the execution of TWAP orders.

Recommendations

One way to resolve these issues is to calculate the TWAP weighing entirely on chain, using the oracle prices and volume data obtained via other oracles. However, this has the downside of broadcasting the volume and nature of future orders, making the potential payoff for any price-manipulation attacks more deterministic and reliable.

We recommend continuing to calculate the TWAP suborders off chain, while also refactoring the on-chain logic so that it only attempts to constrain the aspects of TWAP suborders that it actually can and does successfully constrain. For instance, as described above, the `subOrderCount` constraint does not successfully limit the number of suborders associated with a TWAP order, so for this, we recommend either removing the constraint entirely or adding functionality to use the `subOrderIndex` like a nonce, so that a different suborder `orderHash` cannot use the same index on the same parent.

Similarly, consider adding a fee field to the struct of the TWAP order and the requisite business logic for verifying it through the signature and enforcing it.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

Acknowledged as Design Choice: TWAP sub-orders are intentionally not user-signed. Users cannot predict future prices when signing, so price constraints must be enforced off-chain by the Sequencer. This follows the same trust model as the rest of the system — users trust the Sequencer for fair execution. Price fairness is guaranteed through oracle validation and off-chain matching engine controls, not on-chain signatures.

3.10. The RemoveIsolatedMargin transaction does not recognize collateral added through funding

Target	PositionManager		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The RemoveIsolatedMargin transaction removes the collateral from the isolated position and adds it to the cross-account position. The contract is responsible for ensuring that removing the collateral does not make the isolated position risky. The updatePosition function performs this check, and the getMaxRemovableIsolatedMargin function provides the way to check the maximum amount to be removed (and it is also checked in the RemoveIsolatedMargin transaction).

However, the getMaxRemovableIsolatedMargin function does not recognize the changes on the collateral due to the accumulated funding.

Impact

If the isolated position gained collateral through the funding, trying to remove it using the RemoveIsolatedMargin transaction may revert.

Recommendations

Consider adding the logic applying the funding before calculating the maximum amount to be removed from the isolated position.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [c1f704bc](#).

3.11. Using the median oracle price's timestamp for the aggregate timestamp leads to undesirable edge-case behavior

Target	OraclePriceTickTransLib		
Category	Business Logic	Severity	Low
Likelihood	High	Impact	Low

Description

In the OraclePriceTickTransLib contract, every time the oracle ticks come in, the median data point is calculated and then its timestamp is used to set the price used by the rest of the system:

```
function _processSingle(OraclePricesTick memory tick, IOracle oracle,
    GeneralConfig generalConfig) private {
    // [...]
    OraclePrice memory median = medianPrice(pricesArray, timestamps,
        validCount);
    // [...]
    oracle.updatePrice(tick.assetId, convertedMedian);
}
```

```
function medianPrice(int256[] memory pricesArray, uint256[] memory timestamps,
    uint256 count)
    internal
    pure
    returns (OraclePrice memory)
{
    require(count > 0, "OraclePriceTickTransLib: no valid prices");

    _sortAscending(pricesArray, timestamps, count);

    uint256 mid = count / 2;
    if (count % 2 == 0) {
        // For even count, average the two middle values
        int256 avgPrice = (pricesArray[mid - 1] + pricesArray[mid]) / 2;
        // Use the timestamp of the upper middle element
        return OraclePrice({price: avgPrice, timestamp: timestamps[mid]});
    }
    return OraclePrice({price: pricesArray[mid], timestamp: timestamps[mid]});
}
```

However, in the Oracle contract, this new price is rejected if its timestamp is before the previous update's timestamp:

```
function updatePrice(uint64 assetId, OraclePrice memory newPrice)
    external
    override
    onlyTransactionProcessorOrOwner
{
    require(
        newPrice.timestamp > prices[assetId].timestamp,
        "Oracle: new price timestamp must be greater than current price
        timestamp"
    );
    require(newPrice.price > 0, "Oracle: price negative");
    prices[assetId] = newPrice;
    emit UpdatePrice(assetId, newPrice);
}
```

Furthermore, during the `_validatePrice` procedure, each price entry is compared against the `currentTimestamp`, which is the timestamp of the median of the last tick:

```
function _validatePrice(
    // [...]
) private view {
    // [...]
    require(priceEntry.timestamp > currentTimestamp, "OraclePriceTickTransLib:
    timestamp not increasing");
    // [...]
}
```

The logic where the aggregate price's timestamp is copied from the actual median price point likely leads to undesirable edge-case behavior. When the median is taken, the choice of which actual price to take will aggregate information from all of the other prices. So the timestamp of the median price may actually be significantly lower than the existing timestamp, while the price still reflects newer information. This causes the check in `_validatePrice` to be more lax than intended.

For example, say that oracle price ticks try to happen once per minute, and we have 10 oracles. Eight of the 10 oracles inconsistently update around once every minute, and the other two oracles are very slow but consistent, updating only once an hour. If only the fast oracles were enabled, then sometimes oracles would be correctly excluded from the median because they did not update since the previous tick. The same would happen in most cases if we add in the slow oracles — except if a slow oracle happens to be the median price. If a slow oracle's price, with a timestamp far in the past, is the median, then the next tick will have a far more lax time window, because the comparison in `_validatePrice` will compare all of the next tick's update timestamps against that slow timestamp. So, for one or more future ticks, all of the fast oracles will be used regardless of how many minutes they lagged behind.

In addition to the above effect, note that the check in `updatePrice` for timestamp consistency will always pass. This is because the checks in `_validatePrice` ensure that every timestamp that goes into the median already passes that check, so the median would of course also pass the check. This repeated logic is a sign that the intended logic is not being correctly implemented here.

Impact

The exclusion of oracles due to the timestamp check in `_validatePrice` causing the sequencer to drop that specific oracle's signature (lest the entire transaction be reverted) will be done inconsistently due to the timestamp of the median price being used as the overall timestamp.

Recommendations

It is unclear what the intended design of the timestamp-based exclusion logic is.

In addition to the above, also note that the check in `updatePrice` for timestamp consistency will always pass. This is because the checks in `_validatePrice` ensure that every timestamp that goes into the median already passes that check, so the median would of course also pass the check. So removing this check will not change the logic as is.

On the other hand, removing just the timestamp check from `_validatePrice` would still yield unintentional edge-case behavior. For example, say that we have removed this check, and there are now five oracles. Two of these oracles, A and B, tick slowly, and the other three, X, Y, and Z, tick quickly. At one tick, sorting the prices yields the order XYABZ, and A's timestamp is newer than B's timestamp. For this tick, the median price is A's price, and the timestamp would be set to A's timestamp. At the next tick, XYZ have gotten updated prices and timestamps, but AB have not, and the new ordering is XABYZ. Now, B's price is the median, so the timestamp passed into `updatePrice` would be B's timestamp. But this timestamp would be rejected because it is older, even though the selection of B itself used newer information — namely, the data from XYZ.

Overall, we recommend reviewing the design parameters of the oracles themselves and implementing a timestamp-based staleness check that aligns with those designs.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

The timestamp is bound to its corresponding signed price entry and cannot be separated from it. The only case where the aggregate timestamp diverges from a directly observed data point is when the count of valid prices is even, in which case the price is averaged between the two middle elements while the timestamp of the upper middle element is used. This behavior is an inherent constraint of the current oracle aggregation design and does not have a straightforward mitigation

3.12. Zero orderHash case for order signature check may cause type confusion

Target	EIP712VerificationLib		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In the EIP712VerificationLib contract, `_enforcePermissionsAndNonce` and `_enforceOrderPermissionsAndNonce` both call the `_enforcePermissionsAndNonceInternal` helper function in order to consolidate their logic:

```
function _enforcePermissionsAndNonce(
    // [...]
) private returns (bool) {
    return _enforcePermissionsAndNonceInternal(
        signerManager, accountId, signer, nonce, requiredMask, bytes32(0),
        context
    );
}

function _enforceOrderPermissionsAndNonce(
    // [...]
    bytes32 orderHash,
    // [...]
) private returns (bool) {
    return _enforcePermissionsAndNonceInternal(
        signerManager, accountId, signer, nonce,
        SignerPermissions.ALLOW_SUBMIT_ORDER, orderHash, context
    );
}

function _enforcePermissionsAndNonceInternal(
    // [...]
    bytes32 orderHash,
    // [...]
) private returns (bool) {
    // Check permissions and nonce in one call
    bool isValid;
    if (orderHash != bytes32(0)) {
        isValid =
```

```
        signerManager.checkOrderSignerPermissionsAndNonce(accountId,
        signer, nonce, orderHash, requiredMask);
    } else {
        isValid = signerManager.checkSignerPermissionsAndNonce(accountId,
        signer, nonce, requiredMask);
    }

    if (isValid) {
        return true;
    }
    // [... subsequent logic always reverts]
}
```

Here, a zero orderHash is used as a special value in order to indicate that the object is not an order. But if the `_enforceOrderPermissionsAndNonce` function is called with a zero in its arguments, then the logic will unintentionally use the wrong `checkSignerPermissionsAndNonce` function.

Impact

There is no impact because currently, all of the call sites of `_enforceOrderPermissionsAndNonce` require that the `orderHash` be the output of a hash, which would not be zero.

Despite the lack of impact, this is still a footgun for future versions of the code.

Recommendations

We recommend requiring that `orderHash` is not zero in `_enforceOrderPermissionsAndNonce` to ensure that the mismatched flow cannot occur even if a future version of this contract calls this with attacker-supplied unchecked input that could be zero.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

The likelihood of introducing a new on-chain order type that produces a zero `orderHash` is negligible given the current and planned design.

3.13. Changing the mode of risk segment is nonatomic

Target	GeneralConfig		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

For an asset, the risk rate can be configured over the segment of total balance or total value of a position.

```
function getRiskFactor(uint64 assetId, int256 absBalance, int256 price)
    public view returns (int256, bool) {
    SyntheticAssetInfo storage assetInfo = syntheticAssetsInfo[assetId];
    RiskFactorSegment[] storage segments = assetInfo.riskFactorSegments;
    uint256 segmentLen = segments.length;
    if (segmentLen == 0) {
        return (0, false);
    }

    int256 amount = absBalance;
    if (!isRiskByBalanceOnly) {
        amount = price * absBalance;
    }

    for (uint256 i = 0; i < segmentLen;) {
        if (amount <= segments[i].upperBound) {
            return (segments[i].risk, true);
        }
        unchecked {
            ++i;
        }
    }
    return (segments[segmentLen - 1].risk, true);
}
```

Each asset has `riskFactorSegments`, which may indicate the segments of total balance or total value. This field may be set through the `setSyntheticAssetsInfo` function.

The `isRiskByBalanceOnly` global variable decides whether these `riskFactorSegments` fields indicate the segments of total balance or not. This may be toggled through the `setIsRiskByBalanceOnly` function.

Note that these are different functions. These functions should be consecutively invoked in order to correctly calculate the risk factors.

Impact

If the owner does not consecutively invoke these functions, the calculation of risk factor may become problematic between these changes. This can lead to incorrect liquidations and statistics related to protocol solvency and risk.

Recommendations

Ensure that it is possible to invoke these two functions consecutively on the owner. Alternatively, consider refactoring the configuration system to one that does not require these two functions to be invoked consecutively.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

Configuration changes of this nature are governed off-chain. Any change to `isRiskByBalanceOnly` or `riskFactorSegments` requires a coordinated update procedure that includes pausing the sequencer — and suspending service if necessary — to ensure no transactions are processed between the two dependent calls.

3.14. The AddIsolatedMargin transaction can be processed for delisted assets

Target	IsolatedMarginTransLib, PositionManager		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The AddIsolatedMargin transaction allows users to add collateral to an isolated position for a delisted asset.

Impact

It does not affect the integrity or availability of the business logic. Nonetheless, this is the only transaction that allows the position to be created for a delisted asset.

Recommendations

Consider disallowing the addition of collateral to an isolated position for a delisted asset.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [73099647](#).

3.15. Unused asset ID of collateral

Target	*		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The protocol has only one collateral asset, which is stated in the LoadingZone contract and cannot be changed. However, the codebase has variables, fields, and checks referring to the asset ID of the collateral, which seems to not be needed.

Impact

This does not affect the business logic, but it may confuse developers.

Recommendations

Consider removing unused variables, fields, and relevant checks.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

The collateral asset ID fields are a legacy artifact of an earlier design. They are intentionally preserved for backwards compatibility and do not affect correctness.

3.16. The deposit workflow does not support rebasing or fee-on-transfer tokens

Target	LoadingZone		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The LoadingZone contract holds tokens that users deposit before a transaction adds them to an account. In order to keep track of these pending deposits, it maintains a balances mapping that stores the amount deposited per user:

```
function deposit(uint256 amount) external {
    // [...]
    balances[msg.sender] += amount;
    // EdgexVault pulls tokens from user (user must approve EdgexVault)
    edgexVault.depositFrom(msg.sender, address(depositToken), amount);

    emit TokenDeposited(msg.sender, amount, balances[msg.sender]);
}
```

It is important to note that this logic does not support rebasing or fee-on-transfer tokens.

Rebasing tokens will occasionally asynchronously change the balances of all of its holders, which means that this balances mapping will likely be incorrect after the first rebase.

Fee-on-transfer tokens do not increase the balance of the recipient by the same quantity as the ERC-20 transfer function was called with. So, for those tokens, each deposit's fee will be misaccounted and taken out of the pool of pending deposits so that the last one or several withdrawers out of the vault would not be able to finalize their withdrawals.

Impact

Rebasing tokens and fee-on-transfer tokens will break the accounting in subtle ways. Since these tokens are not supported by this protocol, there is effectively no impact.

Recommendations

We recommend documenting the lack of support for rebasing and fee-on-transfer tokens.

Alternatively, if fee-on-transfer tokens should be supported, then we recommend measuring the

balance before and after the call to `depositFrom`, in order to determine how many tokens the vault actually received, and then add that quantity to `balances[msg.sender]`.

Rebasing tokens are more complex. If they are to be supported, then we recommend approaching them on a case-by-case basis. For most rebasing tokens, there exists a way to get a fixed share quantity that is exact but does not rebase.

In the extreme case where arbitrary rebasing tokens must be supported, and these tokens are not necessarily proportionately rebased, then the only completely safe way to track the future value of a user's current tokens is to deploy a holding contract that then owns those tokens. This, however, also complicates the accounting of the actual collateral balance after the deposit is finished, so more of the system would need to be redesigned.

Remediation

This issue has been acknowledged by EdgeX. EdgeX added the following note:

The protocol exclusively supports official USDC and standard ERC-20 tokens with no transfer fees or rebasing mechanics.

3.17. Funding calculation rounding should always round against the user

Target	Funding		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The Funding contract tracks the funding index for each asset, which is a way to continuously transfer an asset's interest rate from the longs to the shorts, or vice versa for unusual assets. See the `positionApplyFunding` function, which is called when pending funding calculations need to be applied:

```
function positionApplyFunding(AccountPosition memory position)
    external view returns (AccountPosition memory) {
        int256 collateralFXP = 0;
        (position, collateralFXP) = _applyPositionInner(position);
        position.collateralBalance = collateralFXP / FXP_32_ONE;
        return position;
    }

function _applyPositionInner(AccountPosition memory position)
    internal
    view
    returns (AccountPosition memory, int256)
{
    int256 currentCollateralFXP = position.collateralBalance * FXP_32_ONE;
    uint256 len = position.assets.length;
    for (uint256 i = 0; i < len;) {
        // [...]
        int256 balance = position.assets[i].balance;
        // [...]

        int256 deltaFundingIndex = globalIndex - cachedIndex;

        int256 fxpDeltaFunding = balance * deltaFundingIndex;
        // [...]
        // Route funding PnL based on margin mode: isolated positions adjust
        // their own isolated bucket.
        if (position.assets[i].packed.getMarginMode() == MARGIN_MODE_ISOLATED)
        {
```

```
        position.assets[i].isolatedCollateral -= fxpDeltaFunding
    / FXP_32_ONE;
    } else {
        currentCollateralFXP -= fxpDeltaFunding;
    }
    unchecked {
        ++i;
    }
}
return (position, currentCollateralFXP);
}
```

The intermediary collateral quantity is calculated in units of FXP_32_ONE, dividing by the fixed point either per isolatedCollateral in the MARGIN_MODE_ISOLATED case or broadly up in positionApplyFunding for the cross-collateralization case.

This division always rounds towards zero. Since the division is used to calculate the amount to deduct from the collateral, this means that for long positions, the rounding is always favorable to the user, and for short positions, it is always against the user.

Impact

These rounding errors are not necessarily balanced, so, over time, this can accumulate dust. Additionally, depending on gas prices and sequencer throughput, attackers may be able to manipulate this rounding in order to gain a small amount of profit through matched delta-neutral positions broken up in different granularities.

Recommendations

We recommend always rounding against the user for these funding calculations.

Remediation

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [e4f2419c](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. If re-deployed, the Create2Factory may be a reputational hazard

The scoped code includes the Create2Factory contract, which appears to be a copy of the standard one that allows for permissionless deterministic contract deployment, and is itself usually permissionlessly deterministically deployed via Nick's Method for keyless single-transaction signing.

This discussion point is not applicable if there is a general Create2Factory for the target chain that EdgeX would just be calling into. However, if EdgeX deploys this contract themselves, then it is important to note that this contract allows anyone to deploy any contract through it:

```
function deploy(bytes memory bytecode, bytes32 salt)
    external payable returns (address deployedAddress) {
    if (bytecode.length == 0) revert EmptyBytecode();

    assembly {
        let data := add(bytecode, 0x20)
        let size := mload(bytecode)
        deployedAddress := create2(callvalue(), data, size, salt)
    }

    if (deployedAddress == address(0)) revert DeploymentFailed();

    emit Deployed(deployedAddress, salt);
}
```

If the factory is deployed by an address that is publicly associated with EdgeX, then attackers may attempt to use this to deploy malicious contracts that falsely appear to be associated with EdgeX. So, in that case, we recommend adding access control to this factory.

EdgeX added the following note:

A new deployment mechanism has been designed. The updated approach introduces an explicit SALT parameter under controlled access.

4.2. Keeping assets sorted by asset ID could be a worthwhile optimization

In many parts of this design, the list of assets must be looped through in order to locate a specific asset. The worst case of this happens in the `_checkSmallerInSyntheticHoldings` function, which loops through every asset for every asset, in order to compare corresponding assets between `initPosition` and `updatedPosition`.

Instead of keeping the list of assets in a position unsorted, we recommend sorting it by asset ID. This would increase the time it takes to add or remove an arbitrary asset from constant to linear time, but that drawback does not functionally increase the running time, because the position checks and write back both must loop through the entire position anyways.

If the list of assets were always sorted, then, in addition to allowing this list to be binary searched for the corresponding asset ID for those single loops, it would also allow `_checkSmallerInSyntheticHoldings` to run in linear time instead of quadratic time, because it can go through both lists once in order. If the assets being compared do not equal, then the smaller asset can safely be concluded to not appear at all in the other list.

EdgeX added the following note:

In practice, the asset lists produced by the initialization and update paths are already in the same deterministic order. Additionally, a priority-matching optimization has been implemented at `src/position/PositionManager.sol:325` that short-circuits the common case.

4.3. Inconsistent hash ordering between `AddIsolatedMargin` and `RemoveIsolatedMargin`

The `AddIsolatedMargin` and `RemoveIsolatedMargin` structs have a hash order where the first three fields are reordered:

```
struct AddIsolatedMargin {
    uint64 accountId; // Account ID
    int256 collateralAmount; // Amount of collateral to move from cross to
    isolated
    uint64 assetId; // Asset ID to add margin to
    uint256 nonce;
    uint256 deadline;
    address signer; // Signer authorizing isolated margin increases
    bytes signature;
}

struct RemoveIsolatedMargin {
    uint64 accountId; // Account ID
```

```
uint64 assetId; // Asset ID to remove margin from
int256 collateralAmount; // Amount of collateral to move from isolated to
cross
uint256 nonce;
uint256 deadline;
address signer; // Signer authorizing isolated margin decreases
bytes signature;
}
```

This ordering matches the order in which they are hashed for the signature and typehash, but, for consistency, it may be better to fix this ordering.

This issue has been acknowledged by EdgeX, and a fix was implemented in commit [0d635d7b](#).

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: Transaction Batchter

Description

The transaction batchter, also referred to as the sequencer, is an off-chain centrally-controlled EOA that receives user input and calls the `processMultiTransactions` function on the `MultiTransactionProcessor` contract with sequences of transactions (transactions in a protocol-defined sense, not EVM transactions) to execute.

Almost all user interaction with this overall system goes through this sequencer. The primary exception is the deposit workflow, which also involves the `Create2Factory` and `LoadingZone` contracts. In contrast to an alternate design where user interaction starts on-chain, this design has several notable points pertaining to its threat model and attack surface:

- The batchter validates some, but not all, of the transaction call parameters off-chain. The correctness of this validation is out of scope for this audit, which focuses only on the on-chain components. The on-chain validation is audited here as a second layer of defense, and the coverage of the findings we identified generally correspond to validation gaps that EdgeX will evaluate in the broader context of the sequencer's off-chain checks.
- Most MEV-style protocol-transaction sequencing attacks do not apply, because attackers who submit transactions through the batchter will not have control over the ordering of their transactions relative to transactions owned by other users, even if they are willing to pay for such re-sequencing.
- Similarly, most MEV-style EVM-transaction sequencing attacks also do not apply, because even though the batchter's call to `processMultiTransactions` itself could be sandwiched, since all of the oracle data ticks go through the batchter, such sandwiching will not change the execution of the EVM-transaction. Although reentrancy is theoretically possible, for the purposes of this audit, we assume that the governance-defined sequencer address is either an EOA without code or a contract that cannot be invoked during attacker-controlled reentrancy.

Transaction types

There are nineteen types of protocol-transactions:

- `TYPE_SET_SIGNER_PERMISSIONS`: Deserializes the payload into a `SetSignerPermissionsTx` struct, and sets the permissions for a specific signer on an

authorized account.

- TYPE_ADD_SIGNERS: Deserializes the payload into an AddSignersWithPermissionsTx struct, and adds signers with the specified permissions on an authorized account.
- TYPE_DEPOSIT: Deserializes the payload into a Deposit struct, and continues processing a deposit that was previously sent to the LoadingZone.
- TYPE_WITHDRAWAL: Deserializes the payload into a Withdrawal struct, and processes a withdrawal from the system balance.
- TYPE_TRANSFER: Deserializes the payload into a Transfer struct, and transfers collateral balance from an authorized account to another account.
- TYPE_SET_MARGIN_PREFERENCE: Deserializes the payload into a SetMarginPreference struct, and sets the margin preference to isolated or cross-collateralized for a synthetic asset type, with the precondition that the user does not have a position in that asset.
- TYPE_ADD_ISOLATED_MARGIN: Deserializes the payload into an AddIsolatedMargin struct, and moves collateral balance from the cross-collateralized position into a specific isolated-margin position.
- TYPE_REMOVE_ISOLATED_MARGIN: Deserializes the payload into a RemoveIsolatedMargin struct, and moves collateral balance from a specific isolated-margin position into the cross-collateralized position.
- TYPE_TRADE: Deserializes the payload into a Trade struct, and fills two orders that the sequencer has identified as appropriate to execute each other.
- TYPE_CREATE_TWAP_ORDER: Deserializes the payload into a CreateTwapOrder struct, and creates a TWAP order, which can then be referred to by sub-orders.
- TYPE_LIQUIDATE: Deserializes the payload into a Liquidate struct, and liquidates a risky position by transferring assets and collateral between the risky account and a liquidator account, while filling an order signed by the liquidator.
- TYPE_DELEVERAGE: Deserializes the payload into a Deleverage struct, and deleverages a risky position by transferring assets and collateral between the risky account and a deleverager account, without any order, in a way that the sequencer has identified as appropriate.
- TYPE_DELIST_SETTLE: Deserializes the payload into a DelistSettle struct, and settles a position while skipping some risk calculations for assets that have been delisted by governance.
- TYPE_FUNDING_TICK: Deserializes the payload into a FundingTick struct, and submits an update to the current funding tick, which is signed by the configured fundingFeeder, in order to transfer effective interest between the longs and the shorts on a synthetic asset.
- TYPE_ORACLE_PRICES_TICK: Deserializes the payload into an OraclePricesTick struct, submits an update to the current oracle price by referencing prices signed by specific permissioned oracles, and then updates the median price.
- TYPE_FORCED_WITHDRAWAL, TYPE_FORCED_TRADE, TYPE_CONDITIONAL_TRANSFER: These transaction types are deprecated, and the audited version of this code reverts if any of these are included in the batch.

6. Assessment Results

During our assessment on the scoped EdgeX V2 contracts, we discovered 17 findings. One critical issue was found. Three were of high impact, five were of medium impact, two were of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.